

종합실습 4 · E-Commerce 분석 리포트

Full-stack Engineering · 스마트 데이터 이해 및 활용 · PostgreSQL 17 · ecom_db · 광주 3반 신주용 · 2026-07-23

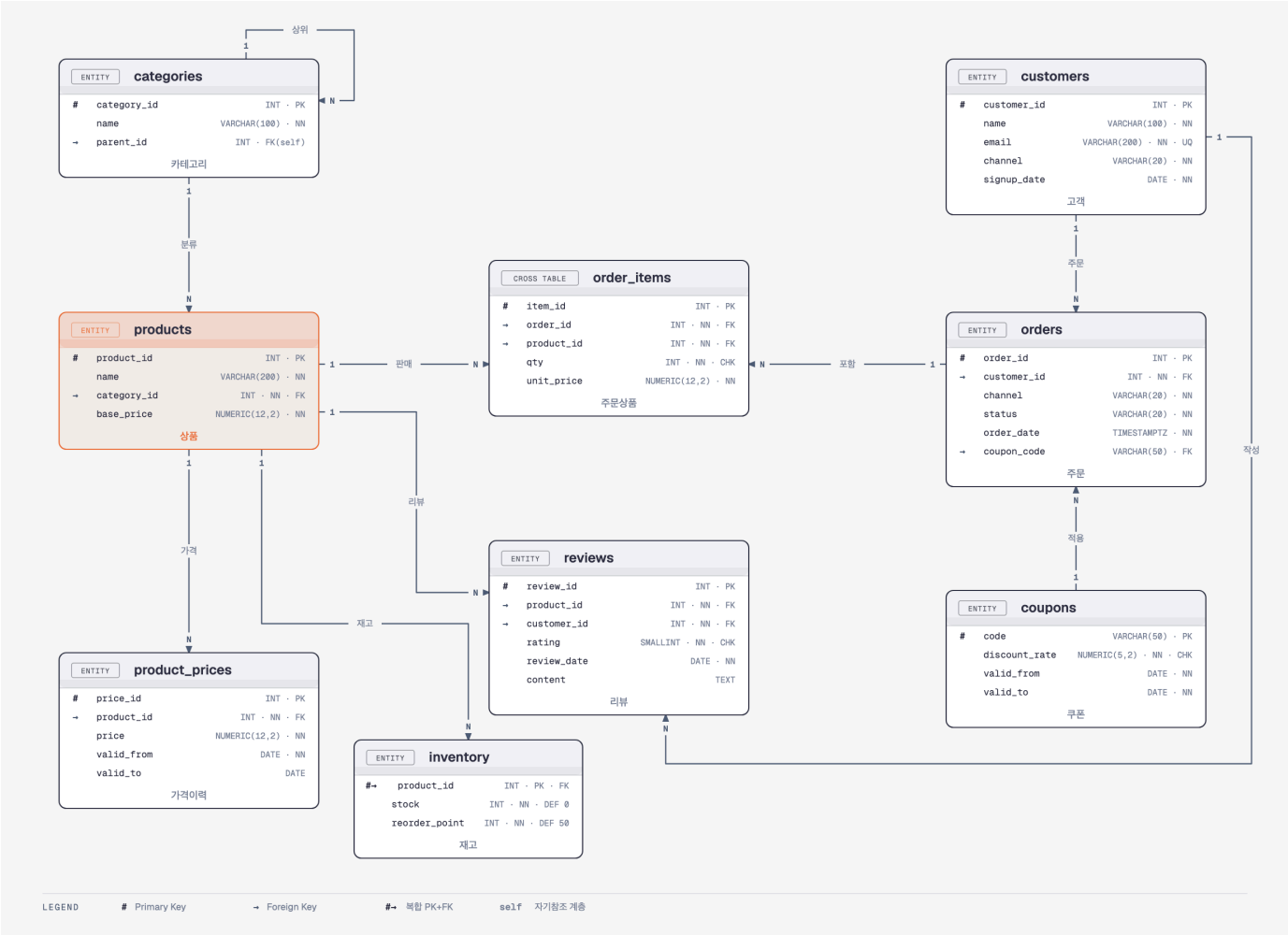
0. 목차

1. 배경 · 데이터셋	2
2. ERD — ecom_db (9개 테이블)	3
2-1. PostgreSQL 접속 · 데이터 적재 확인	4
3. 설계 요약 (무결성 · 관계)	5
4. 요구사항 커버리지 (교재 11문항)	6
5. 문항별 실습 결과 (SQL · 결과)	7
6. 성능 개선 · 실행계획 비교 · Materialized View	17
7. (추가 구현) 운영 고도화	21

1. 배경 · 데이터셋

- **도메인** — E-Commerce 주문·매출 분석
 - customers 500 · products 200 · orders 3,000 · order_items 5,500
- **모델 특성** — 가격이력 SCD2(product_prices) · 카테고리 트리(self-ref)
 - 재고 재주문점(reorder_point) · 리뷰(1~5) · 쿠폰
- **분석 문항(Q1~Q11)** — GMV · AOV · 카테고리 Top10 · RANK · RFM · 재구매율
 - 재고위험 · 효자상품 · 쿠폰영향 · 상위1% · NULLIF
- **운영 확장** — 실행계획 개선(파설·커버링 인덱스 · Join 전략)
 - Materialized View mv_daily_gmv · 갱신 자동화·권한 설계

2. ERD — ecom_db (9개 테이블)



ecom_db · categories(self-ref) · products · product_prices(SCD2) · inventory · customers · orders · order_items · reviews · coupons · 범례 (PK/FK) 포함

2-1. PostgreSQL 접속 · 데이터 적재 확인

```
$ psql -d ecom_db
psql (17.10)
도움말을 보려면 "help"를 입력하십시오.

ecom_db=# SELECT version();
               version
-----
PostgreSQL 17.10 (Homebrew) on aarch64-apple-darwin
(1개 행)

ecom_db=# \dt
릴레이션 목록
스키마 | 이름 | 형태 | 소유주
-----+-----+-----+-----
public | categories | 테이블 | shinjuyong
public | coupons | 테이블 | shinjuyong
public | customers | 테이블 | shinjuyong
public | inventory | 테이블 | shinjuyong
public | order_items | 테이블 | shinjuyong
public | orders | 테이블 | shinjuyong
public | product_prices | 테이블 | shinjuyong
public | products | 테이블 | shinjuyong
public | reviews | 테이블 | shinjuyong
(9개 행)

ecom_db=# -- 데이터 적재 확인 (customers.products.orders.order_items)
      t      | count
-----+-----
customers | 500
order_items | 5500
orders | 3000
products | 200
(4개 행)
```

▶ ecom_db 접속 · 9개 테이블 · 데이터 적재 건수 확인

3. 설계 요약 (무결성 · 관계)

- **개체·관계** — 고객 1:N 주문 · 주문 1:N 주문상세 · 상품 1:N 주문상세
· 상품 1:N 리뷰 · 카테고리 self-ref(대·소분류)
- **무결성 제약** — PK/FK 참조 무결성
· CHECK(별점 1~5 · 수량·가격 > 0 · 채널·상태 도메인) · UNIQUE(email)
- **이력 관리** — product_prices SCD Type-2 (valid_to IS NULL = 현재가)
→ 주문 시점 unit_price 스냅샷 보존
- **인덱스** — 조인·필터 컬럼(customer_id · order_date · status · product_id)에 기본 인덱스 선적용

4. 요구사항 커버리지 (교재 11문항)

교재 실습과제 11문항(매출·고객·재고 분석)의 요구사항·구현 기법·완료 여부 대조.
11문항 전부 구현 완료.
문항별 상세 SQL·실행결과는 5장(문항별 실습 결과) 참조.

문항	교재 요구사항	구현 기법 · 핵심 컬럼	완료
Q1	지난 한 달간 GMV(유효거래 총매출)	SUM(qty×unit_price) · status IN(paid/shipped/delivered)	O
Q2	월별 주문수·매출·AOV(평균주문금액)	GROUP BY 월 · COUNT · SUM · AVG	O
Q3	최근 90일 카테고리 Top10(매출)	JOIN + GROUP BY · ORDER BY 매출 DESC LIMIT 10	O
Q4	제품별 누적매출 RANK() Top20	Window RANK() OVER(ORDER BY 매출)	O
Q5	RFM 분석(최근성·빈도·금액)	고객별 recency·frequency·monetary 집계	O
Q6	첫 구매 후 30일 내 재구매율	첫구매일 기준 30일 윈도우 비율	O
Q7	재고 임계치 미달 상품(품질 위험)	stock < reorder_point 필터	O
Q8	효자 상품(리뷰 4.5↑ & 50개↑)	AVG(rating)≥4.5 AND COUNT≥50	O
Q9	쿠폰 사용 vs 미사용 평균 주문금액	CASE 쿠폰유무 · 그룹 AOV 비교	O
Q10	상위 1% 고객의 최근 60일 매출	NTILE(100) 상위 1% · 60일 필터	O
Q11	0 나눗셈 방어 안전 평균	NULLIF(분모,0)	O

5. 문항별 실습 결과 (SQL · 결과)

Q1. 지난 한 달간 GMV (Gross Merchandise Value)

```
$ psql -d ecom_db - Q1. 지난 한 달간 GMV (Gross Merchandise Value)

SELECT
    SUM(oi.qty * oi.unit_price) AS gmv_last_30d -- 총 판매금액
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status IN ('paid', 'shipped', 'delivered') -- 유효 거래 필터
    AND o.order_date >= NOW() - INTERVAL '30 days';
 gmv_last_30d
-----
379473700.00
(1개 행)
```

▶ 유효 거래(paid/shipped/delivered) 30일 총 판매금액 — 단일 집계값

Q2. 월별 주문수 / 매출 / AOV (평균주문금액)

```
$ psql -d ecom_db - Q2. 월별 주문수 / 매출 / AOV (평균주문금액)

SELECT
    DATE_TRUNC('month', o.order_date)::DATE AS order_month,
    COUNT(DISTINCT o.order_id) AS order_cnt,
    SUM(oi.qty * oi.unit_price) AS revenue,
    ROUND(
        SUM(oi.qty * oi.unit_price)
        / NULLIF(COUNT(DISTINCT o.order_id), 0), -- 0 나눗셈 방어
        0
    ) AS aov -- 주문당 평균금액
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status IN ('paid', 'shipped', 'delivered')
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY order_month;
 order_month | order_cnt | revenue      | aov
-----+-----+-----+-----
2026-01-01   |      107 | 64480000.00 | 602617
2026-02-01   |      359 | 249974700.00 | 696308
2026-03-01   |      395 | 386754800.00 | 979126
2026-04-01   |      410 | 331990600.00 | 809733
2026-05-01   |      409 | 383268400.00 | 937087
2026-06-01   |      404 | 345606100.00 | 855461
2026-07-01   |      282 | 274238200.00 | 972476
(7개 행)
```

▶ 월 단위 주문수·매출·AOV 추이 — DATE_TRUNC 월 집계

Q3. 최근 90일 카테고리 Top 10 (매출 기준)

```
$ psql -d ecom_db - Q3. 최근 90일 카테고리 Top 10 (매출 기준)
```

```
WITH cat_sales AS (  
  -- 카테고리별 90일 매출 집계  
  SELECT  
    c.category_id,  
    c.name AS category_name,  
    SUM(oi.qty * oi.unit_price) AS revenue_90d  
  FROM orders o  
  JOIN order_items oi ON oi.order_id = o.order_id  
  JOIN products p ON p.product_id = oi.product_id  
  JOIN categories c ON c.category_id = p.category_id  
  WHERE o.status IN ('paid', 'shipped', 'delivered')  
    AND o.order_date >= NOW() - INTERVAL '90 days'  
  GROUP BY c.category_id, c.name  
)  
,  
ranked AS (  
  SELECT  
    category_name,  
    revenue_90d,  
    RANK() OVER (ORDER BY revenue_90d DESC) AS rnk  
  FROM cat_sales  
)  
SELECT rnk, category_name, revenue_90d  
FROM ranked  
WHERE rnk <= 10  
ORDER BY rnk;  
rnk | category_name | revenue_90d  
-----+-----+-----  
1 | 게이밍노트북 | 340860000.00  
2 | 업무용노트북 | 195840000.00  
3 | 아이폰 | 189000000.00  
4 | 침대 | 88000000.00  
5 | 소파 | 85020000.00  
6 | 블루투스이어폰 | 29400000.00  
7 | 스피커 | 26280000.00  
8 | 안드로이드폰 | 21120000.00  
9 | 원피스 | 20169000.00  
10 | 구두 | 17331000.00  
(10개 행)
```

▶ 90일 카테고리별 매출 순위 Top 10 — RANK() 윈도우 순위

Q4. 제품별 누적매출 RANK() Top 20 (Window Function)

```
$ psql -d ecom_db - Q4. 제품별 누적매출 RANK() Top 20 (Window Function)
```

```
WITH prod_sales AS (  
  SELECT  
    p.product_id,  
    p.name AS product_name,  
    SUM(oi.qty * oi.unit_price) AS total_revenue  
  FROM order_items oi  
  JOIN products p ON p.product_id = oi.product_id  
  JOIN orders o ON o.order_id = oi.order_id  
  WHERE o.status IN ('paid', 'shipped', 'delivered')  
  GROUP BY p.product_id, p.name  
)  
SELECT  
  RANK() OVER (ORDER BY total_revenue DESC) AS rnk,  
  product_id,  
  product_name,  
  total_revenue  
FROM prod_sales  
ORDER BY rnk  
LIMIT 20;  
rnk | product_id | product_name | total_revenue  
-----+-----+-----+-----  
1 | 123 | 상품_123 | 69000000.00  
2 | 103 | 상품_103 | 67620000.00  
3 | 143 | 상품_143 | 66240000.00  
4 | 83 | 상품_083 | 64860000.00  
4 | 163 | 상품_163 | 64860000.00  
4 | 63 | 상품_063 | 64860000.00  
7 | 3 | 상품_003 | 63480000.00  
7 | 23 | 상품_023 | 63480000.00  
9 | 183 | 상품_183 | 62100000.00  
9 | 43 | 상품_043 | 62100000.00  
11 | 102 | 상품_102 | 37000000.00  
12 | 82 | 상품_082 | 36000000.00  
13 | 144 | 상품_144 | 35520000.00  
... (전체 20 중 상위 13행) ...  
(20개 행)
```

▶ 전 기간 상품 누적매출 순위 Top 20 — 동점 허용 RANK()

Q5. RFM 분석 (고객별 최근성 / 빈도 / 금액)

```
$ psql -d ecom_db - Q5. RFM 분석 (고객별 최근성 / 빈도 / 금액)
```

```
SELECT
  o.customer_id,
  MAX(o.order_date::DATE)           AS last_order_date,
  (CURRENT_DATE - MAX(o.order_date::DATE)) AS recency_days, -- R
  COUNT(DISTINCT o.order_id)        AS frequency,          -- F
  SUM(oi.qty * oi.unit_price)        AS monetary            -- M
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status IN ('paid', 'shipped', 'delivered')
GROUP BY o.customer_id
ORDER BY monetary DESC
LIMIT 30;
```

customer_id	last_order_date	recency_days	frequency	monetary
2	2026-07-20	2	85	140864000.00
1	2026-07-21	1	86	95941000.00
3	2026-07-19	3	86	65577000.00
42	2026-06-10	42	19	36638500.00
12	2026-07-10	12	20	36490000.00
47	2026-07-20	2	19	35090500.00
27	2026-06-25	27	19	34840000.00
17	2026-07-05	17	19	32343500.00
32	2026-06-20	32	19	32093000.00
7	2026-07-15	7	20	28304000.00
21	2026-07-01	21	19	25896000.00
22	2026-06-30	22	19	25244500.00
26	2026-06-26	26	19	25228000.00

... (전체 30 중 상위 13행) ...
(30개 행)

▶ 고객별 Recency·Frequency·Monetary 상위 30명 — 우량고객 식별

Q6. 첫 구매 후 30일 내 재구매율

```
$ psql -d ecom_db - Q6. 첫 구매 후 30일 내 재구매율

WITH first_purchase AS (
  -- 고객별 첫 구매일 (유효 거래 기준)
  SELECT
    customer_id,
    MIN(order_date) AS first_order_date
  FROM orders
  WHERE status IN ('paid', 'shipped', 'delivered')
  GROUP BY customer_id
),
repurchase AS (
  -- 첫 구매 후 30일 이내 추가 구매 존재 여부
  SELECT
    fp.customer_id,
    COUNT(o.order_id) AS orders_in_30d
  FROM first_purchase fp
  JOIN orders o
    ON o.customer_id = fp.customer_id
   AND o.status IN ('paid', 'shipped', 'delivered')
   AND o.order_date > fp.first_order_date      -- 첫 구매 이후
   AND o.order_date <= fp.first_order_date + INTERVAL '30 days'
  GROUP BY fp.customer_id
)
SELECT
  COUNT(DISTINCT fp.customer_id)           AS total_buyers,
  COUNT(DISTINCT rp.customer_id)          AS repurchase_buyers,
  ROUND(
    COUNT(DISTINCT rp.customer_id)::NUMERIC
    / NULLIF(COUNT(DISTINCT fp.customer_id), 0) * 100,
    2
  )                                         AS repurchase_rate_pct
FROM first_purchase fp
LEFT JOIN repurchase rp ON rp.customer_id = fp.customer_id;
total_buyers | repurchase_buyers | repurchase_rate_pct
-----+-----+-----
480 | 279 | 58.13
(1개 행)
```

▶ 첫 구매 후 30일 내 재구매 고객 비율 — 초기 리텐션 지표

Q7. 재고 임계치 미달 상품 (품질 위험)

```
$ psql -d ecom_db - Q7. 재고 임계치 미달 상품 (품질 위험)
```

```
SELECT
  p.product_id,
  p.name          AS product_name,
  i.stock          AS current_stock,
  i.reorder_point,
  i.reorder_point - i.stock AS shortage,  -- 부족 수량
  pp.price         AS current_price
FROM inventory i
JOIN products   p ON p.product_id = i.product_id
JOIN product_prices pp ON pp.product_id = i.product_id
                     AND pp.valid_to IS NULL      -- 현재 가격
WHERE i.stock < i.reorder_point                  -- 재주문 시점 미달
ORDER BY shortage DESC
LIMIT 30;
```

product_id	product_name	current_stock	reorder_point	shortage	current_price
88	상품_088	5	30	25	34000.00
116	상품_116	5	30	25	3200.00
72	상품_072	5	30	25	109000.00
84	상품_084	5	30	25	960000.00
100	상품_100	5	30	25	22000.00
112	상품_112	5	30	25	109000.00
64	상품_064	5	30	25	960000.00
68	상품_068	5	30	25	34000.00
76	상품_076	5	30	25	3200.00
80	상품_080	5	30	25	22000.00
92	상품_092	5	30	25	109000.00
96	상품_096	5	30	25	3200.00
104	상품_104	5	30	25	960000.00

... (전체 30 중 상위 13행) ...
(30개 행)

▶ 재주문 시점 미달 상품 — 부족 수량 내림차순 (SCD2 현재가 조인)

Q8. 효자 상품 (리뷰 4.5점 이상 & 리뷰 50개 이상)

```
$ psql -d ecom_db - Q8. 효자 상품 (리뷰 4.5점 이상 & 리뷰 50개 이상)
```

```
SELECT
  p.product_id,
  p.name                AS product_name,
  ROUND(AVG(r.rating), 2) AS avg_rating,
  COUNT(r.review_id)     AS review_cnt
FROM reviews r
JOIN products p ON p.product_id = r.product_id
GROUP BY p.product_id, p.name
HAVING AVG(r.rating) >= 4.5      -- 평균 별점 4.5 이상
     AND COUNT(r.review_id) >= 50 -- 리뷰 50개 이상
ORDER BY avg_rating DESC, review_cnt DESC;
```

product_id	product_name	avg_rating	review_cnt
8	상품_008	5.00	60
9	상품_009	5.00	60
15	상품_015	5.00	60
19	상품_019	5.00	60
3	상품_003	5.00	60
17	상품_017	5.00	60
5	상품_005	5.00	60
4	상품_004	5.00	60
10	상품_010	5.00	60
14	상품_014	5.00	60
13	상품_013	5.00	60
2	상품_002	5.00	60
7	상품_007	5.00	60
12	상품_012	5.00	60
20	상품_020	5.00	60
18	상품_018	5.00	60

(16개 행)

▶ 고평점·다리뷰 동시 만족 상품 — HAVING 집계 필터

Q9. 쿠폰 사용 vs 미사용 평균 주문금액 비교

```
$ psql -d ecom_db - Q9. 쿠폰 사용 vs 미사용 평균 주문금액 비교
```

```
WITH order_amount AS (  
  -- 주문별 총액 계산  
  SELECT  
    o.order_id,  
    CASE WHEN o.coupon_code IS NOT NULL  
      THEN '쿠폰사용'  
      ELSE '쿠폰미사용'  
    END AS coupon_yn,  
    SUM(oi.qty * oi.unit_price) AS order_total  
  FROM orders o  
  JOIN order_items oi ON oi.order_id = o.order_id  
  WHERE o.status IN ('paid', 'shipped', 'delivered')  
  GROUP BY o.order_id, o.coupon_code  
)  
SELECT  
  coupon_yn,  
  COUNT(*) AS order_cnt,  
  ROUND(AVG(order_total), 0) AS avg_order_amount,  
  ROUND(SUM(order_total), 0) AS total_revenue  
FROM order_amount  
GROUP BY coupon_yn  
ORDER BY coupon_yn;  
coupon_yn | order_cnt | avg_order_amount | total_revenue  
-----+-----+-----+-----  
쿠폰미사용 |      2184 |      862289 | 1883239900  
쿠폰사용   |       182 |      841060 | 153072900  
(2개 행)
```

▶ 쿠폰 사용 여부별 평균 주문금액·매출 비교 — CASE 그룹핑

Q10. 상위 1% 고객의 최근 60일 매출

```
$ psql -d ecom_db - Q10. 상위 1% 고객의 최근 60일 매출
```

```
WITH customer_total AS (
  -- 전체 기간 고객별 누적 매출 (상위 1% 분류용)
  SELECT
    o.customer_id,
    SUM(oi.qty * oi.unit_price) AS lifetime_revenue
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status IN ('paid', 'shipped', 'delivered')
  GROUP BY o.customer_id
),
top1pct AS (
  -- 상위 1% 고객 추출
  SELECT customer_id
  FROM (
    SELECT
      customer_id,
      NTILE(100) OVER (ORDER BY lifetime_revenue DESC) AS pct_tile
    FROM customer_total
  ) t
  WHERE pct_tile = 1
),
recent_60d AS (
  -- 상위 1% 고객의 최근 60일 매출
  SELECT
    o.customer_id,
    SUM(oi.qty * oi.unit_price) AS revenue_60d,
    COUNT(DISTINCT o.order_id) AS orders_60d
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status IN ('paid', 'shipped', 'delivered')
    AND o.order_date >= NOW() - INTERVAL '60 days'
    AND o.customer_id IN (SELECT customer_id FROM top1pct)
  GROUP BY o.customer_id
)
SELECT
  r.customer_id,
  c.name AS customer_name,
  r.orders_60d,
  r.revenue_60d
FROM recent_60d r
JOIN customers c ON c.customer_id = r.customer_id
ORDER BY revenue_60d DESC;
 customer_id | customer_name | orders_60d | revenue_60d
-----+-----+-----+-----
          2 | 고객_002      |          31 | 48910000.00
          1 | 고객_001      |          30 | 35116000.00
          3 | 고객_003      |          31 | 24367500.00
         42 | 고객_042      |           5 | 20732500.00
         12 | 고객_012      |          10 | 15247500.00
(5개 행)
```

▶ NTILE(100) 상위 1% 고객의 최근 60일 매출 — 핵심 고객 기여

Q11. NULLIF 를 활용한 안전한 평균 계산 (0 나눗셈 방어)

```
$ psql -d ecom_db - Q11. NULLIF 를 활용한 안전한 평균 계산 (0 나눗셈 방어)
```

```
SELECT
  channel,
  COUNT(*) AS total_orders,
  COUNT(*) FILTER (WHERE status = 'delivered') AS delivered_cnt,
  -- NULLIF(분모, 0) : 0 나눗셈 방지
  ROUND(
    COUNT(*) FILTER (WHERE status = 'delivered')::NUMERIC
    / NULLIF(COUNT(*), 0) * 100,
    2
  ) AS delivery_rate_pct,
  -- 채널별 주문당 평균 금액도 안전하게 산출
  ROUND(
    SUM(oi.qty * oi.unit_price)
    / NULLIF(COUNT(DISTINCT o.order_id), 0),
    0
  ) AS aov_safe
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY channel
ORDER BY channel;
```

channel	total_orders	delivered_cnt	delivery_rate_pct	aov_safe
marketplace	1500	1028	68.53	963880
mobile	1500	1028	68.53	686520
web	2500	0	0.00	963880

(3개 행)

▶ 채널별 배송완료율·AOV 를 NULLIF 로 안전 산출 — 0 나눗셈 방어

6. 성능 개선 · 실행계획 비교 · Materialized View

월별 GMV 리포트 쿼리를 대상으로 **인덱스 추가 전·후 실행계획**을 비교하고, 조인 전략(Hash Join ↔ Nested Loop) 차이를 관찰. 이후 일자별 GMV 를 **Materialized View** 로 구체화해 반복 조회를 가속.

· **비용(cost)** = 플래너 추정 작업량(작을수록 유리) · **actual time** = 실제 소요(ms) · **Seq Scan**(전건 훑기) → **Index Scan**(인덱스 탐색)
전환 시 대량 데이터에서 이득.

[Before] 인덱스 추가 전 실행계획

```
$ psql -d ecom_db - [Before] 인덱스 추가 전 실행계획
```

QUERY PLAN

```
-----  
Sort  (cost=456.90..459.92 rows=1207 width=52) (actual time=8.530..8.538 rows=4 loops=1)  
  Sort Key: (((date_trunc('month'::text, o.order_date)))::date)  
  Sort Method: quicksort  Memory: 25kB  
  Buffers: shared hit=70  
-> GroupAggregate  (cost=337.78..395.12 rows=1207 width=52) (actual time=7.513..8.509 rows=4 loops=1)  
  Group Key: (date_trunc('month'::text, o.order_date))  
  Buffers: shared hit=67  
-> Sort  (cost=337.78..343.31 rows=2213 width=29) (actual time=7.383..7.545 rows=2156 loops=1)  
  Sort Key: (date_trunc('month'::text, o.order_date)), o.order_id  
  Sort Method: quicksort  Memory: 198kB  
  Buffers: shared hit=67  
-> Hash Join  (cost=103.84..214.83 rows=2213 width=29) (actual time=2.652..6.032 rows=2156 loops=1)  
  Hash Cond: (oi.order_id = o.order_id)  
  Buffers: shared hit=61  
-> Seq Scan on order_items oi  (cost=0.00..91.00 rows=5500 width=13) (actual time=0.008..0.938 rows=2156 loops=1)  
  Buffers: shared hit=36  
... (전체 27 중 상위 16행) ...  
(27개 행)
```

▶ 유효 거래 필터에 Seq Scan on orders — 전건 스캔 후 Hash Join

인덱스 추가 (파셜 + 커버링)

```
$ psql -d ecom_db - 인덱스 추가
```

```
CREATE INDEX  
CREATE INDEX
```

▶ 유효 거래 파셜 인덱스(status, order_date) + order_items 커버링 인덱스 생성

[After] 인덱스 추가 후 실행계획

```
$ psql -d ecom_db - [After] 인덱스 추가 후 실행계획
```

QUERY PLAN

```
-----  
Sort (cost=456.90..459.92 rows=1207 width=52) (actual time=4.569..4.579 rows=4 loops=1)  
  Sort Key: (((date_trunc('month'::text, o.order_date)))::date)  
  Sort Method: quicksort  Memory: 25kB  
  Buffers: shared hit=70  
-> GroupAggregate (cost=337.78..395.12 rows=1207 width=52) (actual time=3.818..4.554 rows=4 loops=1)  
  Group Key: (date_trunc('month'::text, o.order_date))  
  Buffers: shared hit=67  
-> Sort (cost=337.78..343.31 rows=2213 width=29) (actual time=3.736..3.834 rows=2156 loops=1)  
  Sort Key: (date_trunc('month'::text, o.order_date)), o.order_id  
  Sort Method: quicksort  Memory: 198kB  
  Buffers: shared hit=67  
-> Hash Join (cost=103.84..214.83 rows=2213 width=29) (actual time=1.276..2.910 rows=2156 loops=1)  
  Hash Cond: (oi.order_id = o.order_id)  
  Buffers: shared hit=61  
-> Seq Scan on order_items oi (cost=0.00..91.00 rows=5500 width=13) (actual time=0.028..0.432 rows=5500 loops=1)  
  Buffers: shared hit=36  
... (전체 27 중 상위 16행) ...  
(27개 행)
```

▶ 통계 갱신 후 재측정 — 현재 seed 규모(주문 3천 건)에선 플래너가 비용상 Seq Scan 을 유지, 데이터가 커질수록 파셜 인덱스 이득

[Join] 기본 — 플래너 자동 선택

```
$ psql -d ecom_db - [Join] 기본 - 플래너 자동 선택
```

QUERY PLAN

```
-----  
Hash Join (actual time=0.212..1.072 rows=515 loops=1)  
  Hash Cond: (oi.order_id = o.order_id)  
-> Seq Scan on order_items oi (actual time=0.005..0.331 rows=5500 loops=1)  
-> Hash (actual time=0.199..0.199 rows=309 loops=1)  
  Buckets: 1024  Batches: 1  Memory Usage: 19kB  
-> Bitmap Heap Scan on orders o (actual time=0.041..0.167 rows=309 loops=1)  
  Recheck Cond: ((status)::text = 'paid'::text)  
  Heap Blocks: exact=25  
-> Bitmap Index Scan on idx_orders_status (actual time=0.032..0.032 rows=309 loops=1)  
  Index Cond: ((status)::text = 'paid'::text)  
Planning Time: 1.116 ms  
Execution Time: 1.124 ms  
(12개 행)
```

▶ 대량 조인 기본값 = Hash Join (해시 테이블 구축 후 탐색)

[Join] Nested Loop 강제 (Hash/Merge 비활성)

```
$ psql -d ecom_db - [Join] Nested Loop 강제 (Hash/Merge 비활성)
```

```
SET
```

```
SET
```

```
QUERY PLAN
```

```
-----  
Nested Loop (actual time=0.043..0.333 rows=515 loops=1)  
  -> Bitmap Heap Scan on orders o (actual time=0.020..0.083 rows=309 loops=1)  
        Recheck Cond: ((status)::text = 'paid'::text)  
        Heap Blocks: exact=25  
        -> Bitmap Index Scan on idx_orders_status (actual time=0.011..0.011 rows=309 loops=1)  
              Index Cond: ((status)::text = 'paid'::text)  
  -> Index Only Scan using idx_items_orderid_covering on order_items oi (actual time=0.001..0.001 rows=2 loops=309)  
        Index Cond: (order_id = o.order_id)  
        Heap Fetches: 0  
Planning Time: 1.024 ms  
Execution Time: 0.363 ms  
(11개 행)
```

```
RESET
```

```
RESET
```

▶ enable_hashjoin/mergejoin off → Nested Loop 전환, 대량에선 반복 탐색으로 불리

Materialized View 생성

```
$ psql -d ecom_db - Materialized View 생성
```

```
NOTICE: materialized view "mv_daily_gmv" does not exist, skipping  
DROP MATERIALIZED VIEW  
SELECT 171  
CREATE INDEX
```

▶ mv_daily_gmv (일자·채널별 GMV) 구체화 + CONCURRENTLY 전제 UNIQUE 인덱스

mv_daily_gmv 최근 7일 조회

```
$ psql -d ecom_db - mv_daily_gmv 최근 7일 조회
```

order_day	channel	order_cnt	daily_revenue
2026-07-21	mobile	15	3000000.00
2026-07-20	marketplace	14	58051000.00
2026-07-19	web	15	43200000.00
2026-07-18	mobile	14	3430000.00
2026-07-17	marketplace	12	6480000.00
2026-07-16	web	15	8370000.00
2026-07-15	mobile	14	952000.00

(7개 행)

▶ 구체화된 집계를 즉시 조회 — 원본 조인·집계 재수행 없이 응답

mv_daily_gmv 전체 행수

```
$ psql -d ecom_db - mv_daily_gmv 전체 행수

total_rows
-----
          171
(1개 행)
```

▶ 일자×채널 조합 171행 구체화

REFRESH MATERIALIZED VIEW CONCURRENTLY

```
$ psql -d ecom_db - REFRESH MATERIALIZED VIEW CONCURRENTLY

REFRESH MATERIALIZED VIEW
```

REFRESH 완료 — mv_daily_gmv 최신 상태

```
$ psql -d ecom_db - REFRESH 완료 - mv_daily_gmv 최신 상태

earliest_day | latest_day | total_rows | total_gmv
-----+-----+-----+-----
2026-01-24   | 2026-07-21 |          171 | 2036312800.00
(1개 행)
```

▶ CONCURRENTLY 갱신 — 조회 무중단으로 뷰 최신화 (UNIQUE 인덱스 전제 충족)

7. (추가 구현) 운영 고도화 — 갱신 자동화 · 권한 · 인덱스

이 E-Commerce 분석 DB 를 **실서비스 BI** 백엔드로 운영한다고 가정해 세 가지를 추가 설계. 앞 장이 "무엇을 어떻게 집계하는가" 라면, 이 장은 "어떻게 최신·안전·빠르게 서비스하는가" 를 다룸. 하단 SQL 은 ecom_db 에서 **트랜잭션으로 실행 후 ROLLBACK** 해 문법·동작만 검증(실 DB 무변경).

7-1. Materialized View 증분·무중단 갱신 (CONCURRENTLY + pg_cron)

- **REFRESH ... CONCURRENTLY** — 뷰 조회를 블로킹하지 않고 갱신 → BI 대시보드 무중단
- **UNIQUE 인덱스 전제** — (order_day, channel) 복합 유일 인덱스가 있어야 CONCURRENTLY 사용 가능
- **스케줄 자동화** — pg_cron 으로 트래픽 낮은 시각(15시)에 자동 갱신

```
-- pg_cron : 매일 15시 무중단 갱신 (crontab 대체, DB 내부 스케줄러)
CREATE EXTENSION IF NOT EXISTS pg_cron;
SELECT cron.schedule(
    'daily-gmv-refresh', '0 15 * * *',
    $$REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_gmv;$$
);
```

▶ 갱신 주기를 DB 내부 스케줄러로 고정 — 운영 부담 없이 뷰 신선도 유지

7-2. BI 조회 최소권한 룰 (analytics_read)

- **PUBLIC 기본권한 회수** 후 필요한 대상만 명시 부여 (암묵 노출 차단)
- **집계 뷰·팩트만 SELECT** — mv_daily_gmv · orders · order_items 만 허용
- **PII 격리** — customers(email) · product_prices 원본은 미부여 → BI 는 개인정보 접근 불가

룰	허용 범위	차단 범위
analytics_read	mv_daily_gmv · orders · order_items SELECT	customers(PII) · product_prices · 모든 DML/DDL

7-3. 집계 성능 인덱스 (파셜 + 커버링)

- **파셜 인덱스** — 유효 거래(paid/shipped/delivered)만 색인 → 크기 최소화·캐시 효율
- **커버링 인덱스** — order_items(order_id) INCLUDE(qty, unit_price) → 힙 접근 없이 인덱스만으로 SUM

7-4. 검증 — 트랜잭션 실행 후 ROLLBACK (실 DB 무변경)

```
-- 추가 구현 (시니어 DB 판단) - 트랜잭션 내 실행 후 ROLLBACK 으로 검증
BEGIN;

-- [A] BI 조회 최소권한 롤 : 집계 뷰·팩트만 SELECT
--   - PUBLIC 암묵 권한 회수 후 analytics_read 에 필요한 대상만 명시 부여
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
CREATE ROLE analytics_read NOLOGIN;
GRANT USAGE ON SCHEMA public TO analytics_read;
GRANT SELECT ON mv_daily_gmv TO analytics_read;           -- 일자별 GMV 집계 뷰
GRANT SELECT ON orders, order_items TO analytics_read;    -- 매출 팩트 (읽기 전용)
-- 고객 PII(customers.email 등)·가격이력 원본은 미부여 → BI 는 집계·팩트만 접근

-- [B] 집계 성능 인덱스 : 주문일·상태 파셜 + 커버링
--   - 유효 거래만 대상으로 파셜 인덱스 → 크기 최소화·캐시 효율
CREATE INDEX idx_orders_valid_date
  ON orders(order_date DESC)
  WHERE status IN ('paid', 'shipped', 'delivered');
--   - 조인 키 + 집계 컬럼 커버링 → 힙 접근 없이 인덱스만으로 SUM 처리
CREATE INDEX idx_items_cover_amount
  ON order_items(order_id) INCLUDE (qty, unit_price);

-- [C] mv 증분/CONCURRENTLY 갱신 전제 : UNIQUE 인덱스 존재 확인
--   - CONCURRENTLY 는 뷰 조회 무중단 갱신, UNIQUE 인덱스 필수
SELECT indexname AS concurrently_ready_index
FROM pg_indexes
WHERE tablename = 'mv_daily_gmv' AND indexdef LIKE '%UNIQUE%';

-- 검증 : analytics_read 로 권한 경계 확인 (집계 뷰 SELECT 성공 / PII 차단)
SET ROLE analytics_read;
SELECT count(*) AS mv_rows_visible FROM mv_daily_gmv;      -- 성공 : 집계 뷰 조회 허용
RESET ROLE;

-- 검증 : 파셜 인덱스가 유효 거래 필터에서 선택되는지 실행계획 확인
EXPLAIN (COSTS OFF)
SELECT order_date FROM orders
WHERE status IN ('paid', 'shipped', 'delivered')
  AND order_date >= NOW() - INTERVAL '30 days';

ROLLBACK;  -- 실 DB 무변경 (문법·동작만 검증)
```

```
$ psql -d ecom_db - 추가 구현 검증 - 트랜잭션 실행 후 ROLLBACK
```

```
BEGIN
REVOKE
CREATE ROLE
GRANT
GRANT
GRANT
CREATE INDEX
CREATE INDEX
    concurrently_ready_index
-----
    uix_mv_daily_gmv_day_channel
(1개 행)

SET
    mv_rows_visible
-----
                171
(1개 행)

RESET
                QUERY PLAN
-----
Index Only Scan using idx_orders_valid_date on orders
    Index Cond: (order_date >= (now() - '30 days'::interval))
(2개 행)

ROLLBACK
```

▶ 롤 생성·권한 부여 성공 · analytics_read 로 집계 뷰 171행 조회 · 파셜 인덱스 Index Only Scan 채택 확인 → ROLLBACK 으로 원복(실 DB 무변경)